

Group: Nguyen Minh Khanh & Truong Minh Son

BT1

- Thực hiện 2 vòng lặp (iterations) để tìm cực tiểu của hàm số bằng phương pháp giảm nhanh nhất (Steepest Descent Method), xuất phát từ điểm $x^{(0)} = 0$.
- Tìm nghiệm tối ưu bằng phương pháp giải tích (analytically).

1. Tìm nghiệm tối ưu bằng phương pháp giải tích (Analytical Solution)

Hàm mục tiêu:

$$f(x_1, x_2) = x_1 + \frac{1}{2}x_2 + \frac{1}{2}x_1^2 + x_2^2 + 3$$

Để tìm cực tiểu, ta tìm gradient của hàm số và tìm giá trị của x để nó bằng 0:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 1 + x_1 \\ \frac{1}{2} + 2x_2 \end{bmatrix}$$

Giải hệ phương trình $\nabla f(x) = 0$:

- $1 + x_1 = 0 \implies x_1 = -1$
- $\frac{1}{2} + 2x_2 = 0 \implies 2x_2 = -\frac{1}{2} \implies x_2 = -\frac{1}{4} = -0.25$

Vậy nghiệm tối ưu (điểm cực tiểu) là:

$$x^* = \begin{bmatrix} -1 \\ -0.25 \end{bmatrix}$$

2. Phương pháp giảm nhanh nhất (Steepest Descent Method)

Công thức lặp:

$$x^{(k+1)} = x^{(k)} - \alpha_k \nabla f(x^{(k)})$$

Trong đó:

- $d^{(k)} = -\nabla f(x^{(k)})$ là hướng tìm kiếm.
- α_k là bước nhảy tối ưu, được tìm bằng cách cực tiểu hóa hàm $f(x^{(k)} + \alpha d^{(k)})$ theo α . Trong BT1, α sẽ được tìm bằng Secant method.

Vòng lặp 1 (Iteration 1):

- Điểm bắt đầu: $x^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$
- Tính gradient tại $x^{(0)}$:

$$\nabla f(x^{(0)}) = \begin{bmatrix} 1 + 0 \\ 0.5 + 2(0) \end{bmatrix} = \begin{bmatrix} 1 \\ 0.5 \end{bmatrix}$$

- Hướng tìm kiếm: $d^{(0)} = -\nabla f(x^{(0)}) = \begin{bmatrix} -1 \\ -0.5 \end{bmatrix}$
- Tính bước nhảy tối ưu α_0 :

Ta cần tìm nghiệm của $g(\alpha) = \nabla f(x^{(0)} + \alpha d^{(0)})^T d^{(0)} = 0$.

$$\text{Điểm } x(\alpha) = \begin{bmatrix} -\alpha \\ -0.5\alpha \end{bmatrix}.$$

$$\text{Gradient tại } x(\alpha): \nabla f(x(\alpha)) = \begin{bmatrix} 1 - \alpha \\ 0.5 - \alpha \end{bmatrix}.$$

Hàm đạo hàm theo hướng:

$$g(\alpha) = (1 - \alpha)(-1) + (0.5 - \alpha)(-0.5) = 1.5\alpha - 1.25$$

Chọn 2 điểm khởi tạo cho Secant: $\alpha_0 = 0$ và $\alpha_1 = 1$.

- Tại $\alpha_0 = 0$: $g(0) = -1.25$
- Tại $\alpha_1 = 1$: $g(1) = 1.5(1) - 1.25 = 0.25$

Áp dụng công thức Secant:

$$\alpha_2 = 1 - 0.25 \frac{1 - 0}{0.25 - (-1.25)} = 1 - 0.25 \left(\frac{1}{1.5} \right) = 1 - \frac{1}{6} = \frac{5}{6} \approx 0.8333$$

- Cập nhật điểm mới $x^{(1)}$:

$$x^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - \frac{5}{6} \begin{bmatrix} 1 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -5/6 \\ -5/12 \end{bmatrix} \approx \begin{bmatrix} -0.8333 \\ -0.4167 \end{bmatrix}$$

Vòng lặp 2 (Iteration 2):

- Điểm hiện tại: $x^{(1)} = \begin{bmatrix} -5/6 \\ -5/12 \end{bmatrix}$

- Tính gradient tại $x^{(1)}$:

$$\nabla f(x^{(1)}) = \begin{bmatrix} 1 + (-5/6) \\ 0.5 + 2(-5/12) \end{bmatrix} = \begin{bmatrix} 1/6 \\ 0.5 - 5/6 \end{bmatrix} = \begin{bmatrix} 1/6 \\ -1/3 \end{bmatrix}$$

- Hướng tìm kiếm: $d^{(1)} = -\nabla f(x^{(1)}) = \begin{bmatrix} -1/6 \\ 1/3 \end{bmatrix}$

- Tính bước nhảy tối ưu α_1 :

$$\text{Điểm } x(\alpha) = \begin{bmatrix} -5/6 - \alpha/6 \\ -5/12 + \alpha/3 \end{bmatrix}.$$

$$\text{Gradient tại } x(\alpha): \nabla f(x(\alpha)) = \begin{bmatrix} 1/6 - \alpha/6 \\ -1/3 + 2\alpha/3 \end{bmatrix}.$$

Hàm đạo hàm theo hướng:

$$g(\alpha) = (1/6 - \alpha/6)(-1/6) + (-1/3 + 2\alpha/3)(1/3)$$

$$g(\alpha) = -1/36 + \alpha/36 - 1/9 + 2\alpha/9 = -5/36 + \alpha/4$$

Chọn 2 điểm khởi tạo cho Secant: $\alpha_0 = 0$ và $\alpha_1 = 1$.

- Tại $\alpha_0 = 0$: $g(0) = -5/36$
- Tại $\alpha_1 = 1$: $g(1) = -5/36 + 1/4 = 4/36 = 1/9$

Áp dụng công thức Secant:

$$\alpha_2 = 1 - \frac{1}{9} \frac{1 - 0}{1/9 - (-5/36)} = 1 - \frac{1/9}{1/9 + 5/36} = 1 - \frac{1/9}{4/36 + 5/36} = 1 - \frac{1/9}{9/36} = 1 - \frac{1/9}{1/4} = 1 - \frac{4}{9} = \frac{5}{9} \approx 0.5556$$

- Cập nhật điểm mới $x^{(2)}$:

$$x^{(2)} = \begin{bmatrix} -5/6 \\ -5/12 \end{bmatrix} - \frac{5}{9} \begin{bmatrix} 1/6 \\ -1/3 \end{bmatrix} = \begin{bmatrix} -5/6 - 5/54 \\ -5/12 + 5/27 \end{bmatrix} = \begin{bmatrix} -45/54 - 5/54 \\ -45/108 + 20/108 \end{bmatrix} = \begin{bmatrix} -50/54 \\ -25/108 \end{bmatrix} = \begin{bmatrix} -25/27 \\ -25/108 \end{bmatrix} \approx \begin{bmatrix} -0.9259 \\ -0.2315 \end{bmatrix}$$

Kết luận:

- Sau 2 vòng lặp, nghiệm xấp xỉ thu được là: $x^{(2)} = \begin{bmatrix} -25/27 \\ -25/108 \end{bmatrix} \approx \begin{bmatrix} -0.9259 \\ -0.2315 \end{bmatrix}$
- Nghiệm tối ưu chính xác (giải tích) là: $x^* = \begin{bmatrix} -1 \\ -0.25 \end{bmatrix}$
- Ta thấy sau 2 vòng lặp, kết quả đã tiến rất gần về phía nghiệm tối ưu.

BT2

Kết quả:

```
PS D:\git\course-master-eng\optimizedDecision>
```

Điểm bắt đầu: $x = [0. \ 0.]$, $f(x) = 3.000000$

```
Vòng lặp 0:  $x = [0. \ 0.]$ ,  $f(x) = 3.000000$ ,  $||grad|| = 1.12e+00$ 
Vòng lặp 1:  $x = [-0.83333333 \ -0.41666667]$ ,  $f(x) = 2.479167$ ,  $||grad|| = 3.73e-01$ 
Vòng lặp 2:  $x = [-0.92592593 \ -0.23148148]$ ,  $f(x) = 2.440586$ ,  $||grad|| = 8.28e-02$ 
Vòng lặp 3:  $x = [-0.98765432 \ -0.26234568]$ ,  $f(x) = 2.437729$ ,  $||grad|| = 2.76e-02$ 
Vòng lặp 4:  $x = [-0.99451303 \ -0.24862826]$ ,  $f(x) = 2.437517$ ,  $||grad|| = 6.13e-03$ 
Vòng lặp 5:  $x = [-0.99908551 \ -0.25091449]$ ,  $f(x) = 2.437501$ ,  $||grad|| = 2.04e-03$ 
Vòng lặp 6:  $x = [-0.99959356 \ -0.24989839]$ ,  $f(x) = 2.437500$ ,  $||grad|| = 4.54e-04$ 
Vòng lặp 7:  $x = [-0.99993226 \ -0.25006774]$ ,  $f(x) = 2.437500$ ,  $||grad|| = 1.51e-04$ 
Vòng lặp 8:  $x = [-0.99996989 \ -0.24999247]$ ,  $f(x) = 2.437500$ ,  $||grad|| = 3.37e-05$ 
Vòng lặp 9:  $x = [-0.99999498 \ -0.25000502]$ ,  $f(x) = 2.437500$ ,  $||grad|| = 1.12e-05$ 
Vòng lặp 10:  $x = [-0.99999777 \ -0.24999944]$ ,  $f(x) = 2.437500$ ,  $||grad|| = 2.49e-06$ 
Vòng lặp 11:  $x = [-0.99999963 \ -0.25000037]$ ,  $f(x) = 2.437500$ ,  $||grad|| = 8.31e-07$ 
```

=> Hội tụ sau 11 vòng lặp!

```
=====
KẾT QUẢ CUỐI CÙNG:
Nghiệm tối ưu tìm được:  $x^* = [-0.99999963 \ -0.25000037]$ 
Giá trị hàm mục tiêu:  $f(x^*) = 2.43750000$ 
Nghiệm giải tích chính xác:  $x^* = [-1.0, \ -0.25]$ 
=====
```

FULL CODE:

```
from collections.abc import Callable

import numpy as np

# =====
# 1. PHƯƠNG PHÁP SECANT CHO LINE SEARCH
# =====

# Reused
def secant(
    function: Callable[[float], float],
    x_previous: float,
    x_current: float,
    epsilon: float,
) -> tuple[float, int]:
    iteration = 0

    while True:
        denominator = function(x_current) - function(x_previous)
        if denominator == 0:
            raise ZeroDivisionError("The secant slope is zero.")

        x_next = x_current - function(x_current) * (x_current - x_previous) / denominator
        iteration += 1

        if abs(x_next - x_current) < abs(x_current) * epsilon:
            return x_next, iteration
```

```

        x_previous, x_current = x_current, x_next

def secant_line_search(
    x,
    d,
    gradient: Callable[[np.ndarray], np.ndarray],
    epsilon=1e-6,
):
    """
    Tìm bước nhảy alpha tối ưu bằng phương pháp Secant.
    Bài toán: Tìm alpha sao cho  $g(\alpha) = \text{Grad}_f(x + \alpha*d)^T * d = 0$ 
    """
    def g(alpha):
        # Đạo hàm theo hướng d tại điểm x + alpha*d
        return np.dot(gradient(x + alpha * d), d)

    alpha, _ = secant(
        function=g,
        x_previous=0.0,
        x_current=1.0,
        epsilon=epsilon,
    )
    return alpha

# =====
# 2. THUẬT TOÁN STEEPEST DESCENT
# =====
def steepest_descent_secant(
    x0,
    function: Callable[[np.ndarray], float],
    gradient: Callable[[np.ndarray], np.ndarray],
    epsilon=1e-6,
    max_iter=1000,
):
    """
    Thuật toán Steepest Descent sử dụng Secant cho Line Search.
    """
    x = np.array(x0, dtype=float)
    print(f"Điểm bắt đầu: x = {x}, f(x) = {function(x):.6f}\n")

    for k in range(max_iter):
        # Tính gradient và chuẩn
        grad = gradient(x)
        grad_norm = np.linalg.norm(grad) # ||g^(k)||

        print(f"Vòng lặp {k}: x = {x}, f(x) = {function(x):.6f}, ||grad|| = {grad_norm:.2e}")

        # TIÊU CHÍ DỪNG: ||g^(k)|| <= epsilon
        if grad_norm <= epsilon:
            print(f"\n=> Hội tụ sau {k} vòng lặp!")
            break

        # Xác định hướng tìm kiếm (Steepest Descent)
        d = -grad

        # Tìm bước nhảy alpha bằng phương pháp Secant
        alpha = secant_line_search(x, d, gradient, epsilon=1e-6)

        # Cập nhật điểm mới
        x = x + alpha * d

    return x

# =====
# 3. CHẠY CHƯƠNG TRÌNH
# =====
if __name__ == "__main__":

```

```
def function(x: np.ndarray) -> float:
    """Objective function for this run."""
    x1, x2 = x[0], x[1]
    return x1 + 0.5 * x2 + 0.5 * x1**2 + x2**2 + 3

def gradient(x: np.ndarray) -> np.ndarray:
    """Gradient of the objective function for this run."""
    x1, x2 = x[0], x[1]
    return np.array([1 + x1, 0.5 + 2 * x2])

# Điểm khởi tạo  $x^{(0)} = 0$ 
x0 = [0.0, 0.0]

# Ngưỡng hội tụ epsilon =  $10^{-6}$ 
epsilon = 1e-6

# Gọi hàm tối ưu
x_opt = steepest_descent_secant(x0, function, gradient, epsilon)

print("\n" + "="*40)
print("KẾT QUẢ CUỐI CÙNG:")
print(f"Nghiệm tối ưu tìm được: x* = {x_opt}")
print(f"Giá trị hàm mục tiêu: f(x*) = {function(x_opt):.8f}")
print("Nghiệm giải tích chính xác: x* = [-1.0, -0.25]")
print("="*40)
```